BoardRent

The Board Game Rental Platform

Property and Inventory Management — Software Requirements

March 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Property and Inventory Management system, a platform enabling users to list, rent, and manage board games. It covers entity definitions, user interface pages, API interfaces, and business logic. All requirements are numbered for traceability.

1.2 Scope

The system supports two roles: Owners (users who list board games for rent) and Renters (users who rent listed games). Key capabilities include:

- Board game listing creation, editing, and deletion
- Rental request submission, approval, and tracking
- Automated notifications for key system events
- Public API interfaces for cross-team availability checks

1.3 Definitions

Term	Definition
Owner	A registered user who lists board games for rent.
Renter	A registered user who requests and pays for a board game rental.
Listing	An Owner's published board game available for rent.

Request	A pending inquiry from a Renter for a specific game and time period.
Rental	A confirmed transaction; a game is in a Renter's possession.
Buffer Period	A mandatory 48-hour window after a Rental's End Date for return and inspection.
Active Flag	A boolean attribute on a Listing indicating whether it is publicly visible.
PER_HOUR / PER_DAY / PER_WEEK	Rate type codes defining how the rental price is computed.

2. Entity Definitions

2.1 Game

Owner: Darius Turcu

A Game entity represents a board game listing created by an Owner.

Attribute	Type	Constraints	Description
game_id	Integer	Unique, auto-generated, NOT NULL	Primary key. Assigned by the system on creation.

owner_id	Integer (FK)	NOT NULL, references Users.user_id	Identifies the Owner who created the listing.
name	String	NOT NULL, 5–30 characters, alphanumeric and special characters allowed	Display name of the board game.
price	Float	NOT NULL, > 0	Rental price. Interpreted relative to rate_type.
rate_type	Enum	NOT NULL, one of: PER_HOUR, PER_DAY, PER_WEEK	Defines the billing interval for the rental price.
player_number	Integer	NOT NULL, >= 1	Recommended number of players for the game.
description	String	NOT NULL, 10–500 characters, alphanumeric and special characters allowed	Human-readable description of the game.
image_link	String	NULLABLE	URL or path of the game's uploaded image. Optional field.
active	Boolean	NOT NULL, DEFAULT TRUE	Indicates whether the listing is publicly visible to Renters.

[ENT-GAM-01] The system shall assign a unique, auto-incrementing integer game_id to each new Game entity upon creation.

[ENT-GAM-02] The system shall associate each Game entity with exactly one owner_id, which shall not be modifiable after creation.

[ENT-GAM-03] The system shall enforce that the name field contains between 5 and 30 characters inclusive. Alphanumeric characters and special characters (e.g. hyphens, apostrophes) are permitted.

[ENT-GAM-04] The system shall enforce that price is a positive floating-point number (price > 0).

[ENT-GAM-05] The system shall enforce that rate_type is one of the enumerated values: PER_HOUR, PER_DAY, or PER_WEEK.

[ENT-GAM-06] The system shall enforce that player_number is a positive integer (>= 1).

[ENT-GAM-07] The system shall enforce that the description field contains between 10 and 500 characters inclusive. Alphanumeric characters and special characters are permitted.

[ENT-GAM-08] The image_link field shall be optional. If not provided, the system shall display a default placeholder image.

[ENT-GAM-09] The active field shall default to TRUE upon creation.

2.2 Rental

Owner: Mihai Tira

A Rental entity represents a confirmed transaction in which a Game is in the possession of a Renter.

Attribute	Type	Constraints	Description
rental_id	Integer	Unique, auto-generated, NOT NULL	Primary key.
game_id	Integer (FK)	NOT NULL, references Games.game_id	The rented game.
renter_id	Integer (FK)	NOT NULL, references Users.user_id	The user paying for the rental.
seller_id	Integer (FK)	NOT NULL, references Users.user_id	The Owner providing the game.
start_date	DateTime (UTC)	NOT NULL	Timestamp when the rental period begins.

end_date	DateTime (UTC)	NOT NULL, end_date > start_date	Timestamp when the game must be returned.

[ENT-REN-01] The system shall assign a unique, auto-incrementing integer rental_id to each Rental entity upon creation.

[ENT-REN-02] The system shall enforce that end_date is strictly after start_date for every Rental.

[ENT-REN-03] The system shall enforce a mandatory 48-hour buffer period immediately following each Rental's end_date. No new Rental for the same game_id may have a start_date within this buffer period.

[ENT-REN-04] The seller_id shall equal the owner_id of the referenced Game entity and shall be validated by the system at creation time.

[ENT-REN-05] Rental records shall never be deleted. They constitute the historical transaction log.

2.3 Request

Owner: Maia Tapordei

A Request entity represents a pending rental inquiry from a Renter that has not yet been confirmed or declined by the Owner.

Attribute	Type	Constraints	Description
request_id	Integer	Unique, auto-generated, NOT NULL	Primary key.
game_id	Integer (FK)	NOT NULL, references Games.game_id	The requested game.
renter_id	Integer (FK)	NOT NULL, references Users.user_id	The user requesting the rental.
seller_id	Integer (FK)	NOT NULL, references Users.user_id	The Owner of the game.

start_date	DateTime	NOT NULL	Requested start of the rental period.
end_date	DateTime	NOT NULL, end_date > start_date	Requested end of the rental period.

[ENT-REQ-01] The system shall assign a unique, auto-incrementing integer request_id to each Request entity upon creation.

[ENT-REQ-02] The system shall enforce that end_date is strictly after start_date for every Request.

[ENT-REQ-03] The system shall prevent creation of a Request for a game_id during any period already covered by an existing Rental (including its 48-hour buffer) for the same game.

[ENT-REQ-04] Upon a request being ACCEPTED, the system shall atomically create a Rental entity from the Request data, delete all overlapping PENDING Requests for the same game_id and delete said request. This operation shall be transactional; if any step fails, the entire operation shall be rolled back.

[ENT-REQ-05] Upon being DECLINED, the system shall delete the Request entity.

[ENT-REQ-06] A Renter may cancel (delete) their own PENDING Request at any time.

2.4 Notification

Owner: Miriam Tranca

A Notification entity represents a system-generated alert delivered to a specific user.

Attribute	Type	Constraints	Description
notification_id	Integer	Unique, auto-generated, NOT NULL	Primary key.
user_id	Integer (FK)	NOT NULL, references Users.user_id	The recipient of the notification.

timestamp	DateTime	NOT NULL, set at creation time	Date and time the notification was generated.
title	String	NOT NULL, max 100 characters	Short summary of the notification event.
body	String	NOT NULL, max 1000 characters	Detailed description of the notification content.

[ENT-NOT-01] The system shall assign a unique, auto-incrementing integer notification_id to each Notification entity upon creation.

[ENT-NOT-02] The timestamp field shall be set by the system at the moment the notification is generated and shall not be modifiable by users.

[ENT-NOT-03] Notification records shall not be deleted by automated processes. Users may dismiss notifications through the UI, but the record shall remain stored.

3. User Interface Requirements

3.1 Menu Bar

Owner: Darius Turcu

[UI-NAV-01] The system shall display a persistent menu bar on every page of the application.

[UI-NAV-02] The menu bar shall contain exactly the following navigable items, in order: Listings, Others' Requests, Others' Rentals, My Requests, My Rentals, Notifications.

[UI-NAV-03] Clicking any menu bar item shall immediately redirect the user to the corresponding page.

[UI-NAV-04] The menu bar shall visually distinguish the currently active page (e.g., via highlighting, underline, or bold styling) so it is unambiguous to the user which page is active.

3.2 Listings Page

Owner: Darius Turcu

[UI-LST-01] The system shall provide a Listings page displaying all Game entities for which owner_id equals the currently authenticated user's user_id.

[UI-LST-02] Each listing entry shall display: the game name, the game picture (or a default placeholder if none is uploaded), and the listing description.

[UI-LST-03] Each listing entry shall include a Delete button. Clicking it shall prompt the user for confirmation before permanently deleting the Game entity and all associated active Requests. Existing Rental records shall not be deleted (per ENT-REN-05).

[UI-LST-04] Each listing entry shall include an Edit button. Clicking it shall redirect the user to the Edit Game page for the selected game.

[UI-LST-05] The page shall include a Create Game button that redirects the user to the Create Game page.

3.3 Create Game Page

Owner: Darius Turcu

[UI-CRG-01] The system shall provide a Create Game page accessible via the Create Game button on the Listings page.

[UI-CRG-02] The page shall present a form containing the fields defined in the table below. All fields marked as mandatory must be completed before submission is permitted.

Field	Type	Mandatory	Constraints
Name	Text input	Yes	5–30 characters. Alphanumeric and special characters allowed.
Picture	Image upload	No	Optional. If provided, the system shall store and display the image.
Price	Numeric input (float)	Yes	Must be > 0.
Rate Type	Dropdown	Yes	Values: PER_HOUR (Per Hour), PER_DAY (Per Day), PER_WEEK (Per Week).

Player Number	Numeric input (integer)	Yes	Must be a positive integer (>= 1).
Description	Text area	Yes	10–500 characters. Alphanumeric and special characters allowed.
Active	Boolean toggle	Yes	Default: TRUE.

[UI-CRG-03] The system shall validate all form inputs against the constraints defined in Section 2.1 (Game entity) before processing the submission.

[UI-CRG-04] If validation fails, the system shall display an inline error message adjacent to each invalid field, identifying the specific constraint violated.

[UI-CRG-05] On successful validation, the system shall create a new Game entity, assign a unique game_id, set owner_id to the currently authenticated user's user_id, and redirect the user to the Listings page.

3.4 Edit Game Page

Owner: Darius Turcu

[UI-EDG-01] The system shall provide an Edit Game page accessible only from the Edit button on a specific listing entry in the Listings page.

[UI-EDG-02] The form shall be pre-populated with the current values of all editable attributes of the selected Game entity.

[UI-EDG-03] The following fields shall be editable: Name, Picture, Price, Rate Type, Player Number, Description, Active. The fields game_id and owner_id shall be read-only and shall not be transmitted in the update request.

[UI-EDG-04] The system shall validate all modified inputs against the same constraints defined in Section 2.1.

[UI-EDG-05] If validation fails, the system shall display an inline error message adjacent to each invalid field.

[UI-EDG-06] On successful validation, the system shall update the corresponding record in the games table and redirect the user to the Listings page.

3.5 Notifications Page

Owner: Miriam Tranca

[UI-NOT-01] The system shall provide a Notifications page displaying all Notification entities where user_id equals the currently authenticated user's user_id.

[UI-NOT-02] Each notification entry shall display: the notification title, the notification body, and the timestamp formatted as YYYY-MM-DD HH:MM (UTC).

[UI-NOT-03] Notifications shall be displayed in descending order by timestamp (most recent first).

[UI-NOT-04] The page shall support pagination. Each page shall display a maximum of 20 notifications. Navigation controls (Previous / Next) shall be displayed when the total count exceeds 20.

[UI-NOT-05] The system shall ensure that only notifications belonging to the authenticated user are retrievable, enforcing access control at the API level.

3.6 Others' Requests Page

Owner: Mihai Tira

[UI-ORQ-01] The system shall provide an Others' Requests page displaying all Request entities where seller_id equals the currently authenticated user's user_id.

[UI-ORQ-02] Each request entry shall display: the game name, the game picture (or default placeholder), the renter's display name, the requested start_date and end_date.

[UI-ORQ-03] Requests shall be displayed in descending order by start_date.

[UI-ORQ-04] Each request entry shall be clickable. Clicking shall redirect the user to the Chat Page associated with that request_id.

3.7 Others' Rentals Page

Owner: Mihai Tira

[UI-ORN-01] The system shall provide an Others' Rentals page displaying all Rental entities where seller_id equals the currently authenticated user's user_id.

[UI-ORN-02] Each rental entry shall display: the game name, the game picture (or default placeholder), the renter's display name, the rental start_date and end_date.

[UI-ORN-03] The list shall display current rentals (where end_date > NOW()) before past rentals (where end_date <= NOW()). Within each group, entries shall be ordered by start_date descending.

3.8 My Requests Page

Owner: Maia Tapordei

[UI-MRQ-01] The system shall provide a My Requests page displaying all Request entities where renter_id equals the currently authenticated user's user_id.

[UI-MRQ-02] Each request entry shall display: the game name, the game picture (or default placeholder), the seller's display name, the requested start_date and end_date, and a Cancel button.

[UI-MRQ-03] Requests shall be displayed in descending order by start_date.

[UI-MRQ-04] Clicking the Cancel button shall prompt the user for confirmation, then delete the Request entity (per ENT-REQ-06).

[UI-MRQ-05] Each request entry shall be clickable (excluding the Cancel button area). Clicking shall redirect the user to the Chat Page associated with that request_id.

3.9 My Rentals Page

Owner: Maia Tapordei

[UI-MRN-01] The system shall provide a My Rentals page displaying all Rental entities where renter_id equals the currently authenticated user's user_id.

[UI-MRN-02] Each rental entry shall display: the game name, the game picture (or default placeholder), the seller's display name, the rental start_date and end_date.

[UI-MRN-03] The list shall display current rentals (where end_date > NOW()) before past rentals (where end_date <= NOW()). Within each group, entries shall be ordered by start_date descending.

3.10 Chat Page

Owner: i(<3)ban team;

Mock Owner: Miriam Tranca

[MOCK-UI-CHT-01] The system shall provide a mocked Chat Page associated with a specific request_id (accessible via the request list pages defined in [UI-ORQ-04] and [UI-MRQ-05]).

[MOCK-UI-CHT-02] The Chat Page shall display exactly two interactive buttons for the Owner: Approve and Deny.

[MOCK-UI-CHT-03] Clicking either the Approve or Deny button shall immediately close the Chat Page and redirect the user back to the previous view.

[MOCK-UI-CHT-04] Upon clicking Approve or Deny, the system shall process and remove the Request entity from the database in accordance with the Request Lifecycle rules defined in Section 5.2 – by calling the before-mentioned API.

4. Application Programming Interfaces (Cross-Team Integration)

This section defines the public API methods exposed by the RentalService component for integration with external teams. All methods are read-only and do not modify system state.

4.1 GetBookedDates

Owner: Luca Taschina

[API-GBD-01] The system shall expose a public method GetBookedDates(gameId, month, year) on the RentalService component.

[API-GBD-02] The gameId parameter is mandatory and must be a valid integer corresponding to an existing Game entity. If the game_id does not exist, the method shall return an empty list.

[API-GBD-03] The month and year parameters are optional. If both are provided, the method shall return only booked date ranges that overlap with the specified calendar month. If omitted, the values will default to the current month, respectively the current year.

[API-GBD-04] The method shall return a list of objects. Each object shall contain: StartDate (DateTime) — the start of the booked interval; EndDate (DateTime) — the end of the booked interval, including the 48-hour buffer period (i.e., rental end_date + 48 hours).

[API-GBD-05] The returned list shall be sorted by StartDate ascending.

4.2 CheckAvailability

Owner: Luca Taschina

[API-CAV-01] The system shall expose a public method CheckAvailability(gameId, startDate, endDate) on the RentalService component.

[API-CAV-02] All three parameters — gameId, startDate, and endDate — are mandatory.

[API-CAV-03] he method shall return a boolean value IsAvailable.T

[API-CAV-04] IsAvailable shall be TRUE if and only if all of the following conditions hold: (a) the requested [startDate, endDate] range does not overlap with any existing Rental interval for the specified game_id; (b) the requested startDate is not within the 48-hour buffer period of any existing Rental; (c) the requested startDate is not more than one month in the future from the current date; (d) the requested endDate is not more than one month in the future from the current date' (e) the game_id corresponds to an existent and active game.

If any condition is not met, IsAvailable shall be FALSE.

[API-CAV-05] The method shall not modify any database state. It is a read-only query.

4.3. CreateRequest

Owner: Maia Tapordei

[API-CRQ-01] The system shall expose a public method CreateRequest(gameId, renterId, startDate, endDate) on the RentalService component.

[API-CRQ-02] The method shall return an Integer value, request_id if positive. Negative values shall denote errors, with the following meanings:

- -1 – "Owner Cannot Rent" error
- -2 – "Dates Unavailable" error
- -3 - "GameID Does Not Exist"

[API-CRQ-03] All four parameters are mandatory. The method shall return "Owner Cannot Rent" if the renterId matches the owner_id of the requested game – an Owner cannot rent their own game.

[API-CRQ-04] If the gameId is not associated to an existent game, the method shall halt and return -3 ("GameID Does Not Exist").

[API-CRQ-05] The system shall validate the requested dates by invoking the CheckAvailability logic. If CheckAvailability returns FALSE, the method shall abort and return the “Dates Unavailable” error.

[API-CRQ-06] The method shall return the auto-generated request_id of the newly created Request entity upon success.

4.4. ApproveRequest

Owner: Mihai Tira

[API-APR-01] The system shall expose a public method ApproveRequest(requestId, ownerId) on the RentalService component.

[API-APR-02] The method shall return an Integer value, representing the newly generated rental_id if positive. Negative values shall denote errors, with the following meanings:

- -1 – "Unauthorized" error
- -2 – "Not Found" error
- -3 – "Transaction Failed" error

[API-APR-03] Both parameters are mandatory. The method shall return the "-2 (Not Found)" error if the requested requestId does not exist. The method shall return the "-1 (Unauthorized)" error if the ownerId does not match the seller_id associated with the Request entity.

[API-APR-04] Upon successful validation, the method shall execute the acceptance lifecycle defined in [BL-LFC-02]. It must perform the following operations atomically as a single database transaction:

(a) Create a new Rental entity using the data from the approved Request.

(b) Delete all other Request entities for the same game_id whose date ranges overlap with the new Rental (including its 48-hour buffer).

(c) Delete the original approved Request entity.

[API-APR-05] If any database operation in the transaction fails, the entire operation shall be rolled back, and the method shall return the "-3 (Transaction Failed)" error.

[API-APR-06] Upon successful execution, the method shall return the auto-generated rental_id and trigger the necessary notifications to any affected Renters whose overlapping requests were cancelled.

4.5. DenyRequest

Owner: Mihai Tira

[API-DNR-01] The system shall expose a public method DenyRequest(requestId, ownerId, reason) on the RentalService component.
[API-DNR-02] The method shall return an Integer value, returning the deleted request_id if positive to indicate success. Negative values shall denote errors, with the following meanings:

- -1 – "Unauthorized" error
- -2 – "Not Found" error

[API-DNR-03] The requestId and ownerId parameters are mandatory; reason is an optional string (max 200 characters). The method shall return the "-2 (Not Found)" error if the requested requestId does not exist.
[API-DNR-04] The method shall return the "-1 (Unauthorized)" error if the ownerId does not match the seller_id associated with the Request entity.
[API-DNR-05] Upon successful validation, the method shall execute the decline lifecycle defined in [BL-LFC-03] by deleting the specified Request entity from the database.
[API-DNR-06] Upon successful execution, the method shall return the original request_id (as a positive integer) and trigger the "Request declined" notification to the Renter, passing the optional reason if provided.

5. Business Logic and Lifecycle

5.1 CRUD Operations

[BL-CRUD-01] The system shall support Create, Read, Update, and Delete operations for Game entities, subject to ownership authorization (only the owner may update or delete their own game).

Owner: Darius Turcu

[BL-CRUD-02] The system shall support Create and Read operations for Rental entities. Rental records shall not be updated or deleted (they are immutable historical records, per ENT-REN-05).

Owner: Mihai Tira

[BL-CRUD-03] The system shall support Create, Read, and Delete operations for Request entities. Update of a Request is performed only through the status lifecycle (Section 5.2).

Owner: Maia Tapordei

[BL-CRUD-04] The system shall support Create and Read operations for Notification entities. Notification records shall not be deleted by the system. Users may dismiss notifications via the UI, which shall mark them as read but retain the record.

Owner: Miriam Tranca

5.2 Request Lifecycle

Owner: Mihai Tira

[BL-LFC-01] A new Request is created. We say it is PENDING while existing in the database.

Owner (Overwrite): Maia Tapordei

[BL-LFC-02] When an Owner sends an ACCEPT signal for a Request: (a) the system shall atomically create a new Rental entity from the Request's data; (b) the system shall delete all other PENDING Requests for the same game_id whose date range overlaps with the accepted rental period (including its 48-hour buffer); (c) the system shall send a notification to each Renter whose overlapping Request was deleted, stating the game is unavailable in their requested period and providing a link to the booking interface; (d) the original Request entity shall be deleted. All steps (a)–(d) must succeed atomically; if any fail, the transaction shall be rolled back.

[BL-LFC-03] When an Owner sends a DECLINE signal for a Request, the system shall delete the Request entity. No Rental is created. The user shall also be notified that their request has been declined.

[BL-LFC-04] A Renter may cancel (delete) their own PENDING Request at any time without requiring Owner approval.

Owner (Overwrite): Maia Tapordei

[BL-LFC-05] Upon a game having its Active flag being set to FALSE, all requests to that game shall be declined and the respective users be notified of the decline.

5.3 Buffer Period Enforcement

Owner: Luca Taschina

[BL-BUF-01] After each Rental's end_date, the system shall enforce a 48-hour buffer period during which no new Rental for the same game_id may start.

[BL-BUF-02] The buffer period shall be enforced at Request creation time, at Request acceptance time, and by the CheckAvailability API (Section 4.2).

[BL-BUF-03] The buffer period shall be represented in GetBookedDates responses by extending each returned EndDate by 48 hours from the actual rental end_date.

5.4 Rental History

Owner: Maia Tapordei

[BL-HST-01] The system shall maintain a permanent historical log of all Rental transactions. Rental records are never deleted.

[BL-HST-02] The Others' Rentals page (Section 3.7) shall display the complete history of games the Owner rented to others, separating current and past rentals.

[BL-HST-03] The My Rentals page (Section 3.9) shall display the complete history of games the Renter rented from others, separating current and past rentals.

5.5 Automated Notifications

Owner: Miriam Tranca

The system shall generate automated Notification entities for the following events:

Event	Recipient	Trigger	Notification Content
New chat message	Message recipient (Owner or Renter)	Immediately upon message send	Title: 'New Message'. Body: sender display name and a link to the Chat Page.
Overlapping request cancelled	Renter whose Request was auto-deleted	When another Request for the same game is accepted (BL-LFC-02)	Title: 'Booking Unavailable'. Body: reason, affected dates, and a link to the booking interface.
Pending requests digest	Owner (seller)	Daily, at 08:00 UTC, for each PENDING Request	Title: 'Unprocessed Rental Request'. Body: game name, renter name, requested dates, and a

		with start_date within 48 hours	link to Others' Requests page.
Upcoming rental reminder	Renter and Owner	24 hours before rental start_date	Title: 'Upcoming Rental Reminder'. Body: game name, rental start_date, and relevant pickup/delivery instructions.
Request declined	Renter	Immediately upon Owner DECLINE signal (BL-LFC-03)	Title: 'Rental Request Declined'. Body: game name, requested dates, and an optional reason if provided by the Owner.

[BL-NOT-01] The system shall generate a Notification entity for each event listed in the table above, addressed to the specified recipient.

[BL-NOT-02] The daily pending-requests digest (row 3) shall be sent only if at least one qualifying PENDING Request exists. No digest shall be sent if there are no qualifying requests.

[BL-NOT-03] The upcoming rental reminder (row 4) shall be sent exactly once per rental, 24 hours (± 5 minutes) before start_date.